

《深度学习》



深度前馈神经网络

内容

神经网络

神经元

网络结构

前馈神经网络

参数学习

计算图与自动微分

优化问题



神经网络

神经网络

神经网络最早是作为一种主要的连接主义模型。

20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing, PDP）网络，其有3个主要特性：

- 1) 信息表示是分布式的（非局部的）；
- 2) 记忆和知识是存储在单元之间的连接上；
- 3) 通过逐渐改变单元之间的连接强度来学习新的知识。

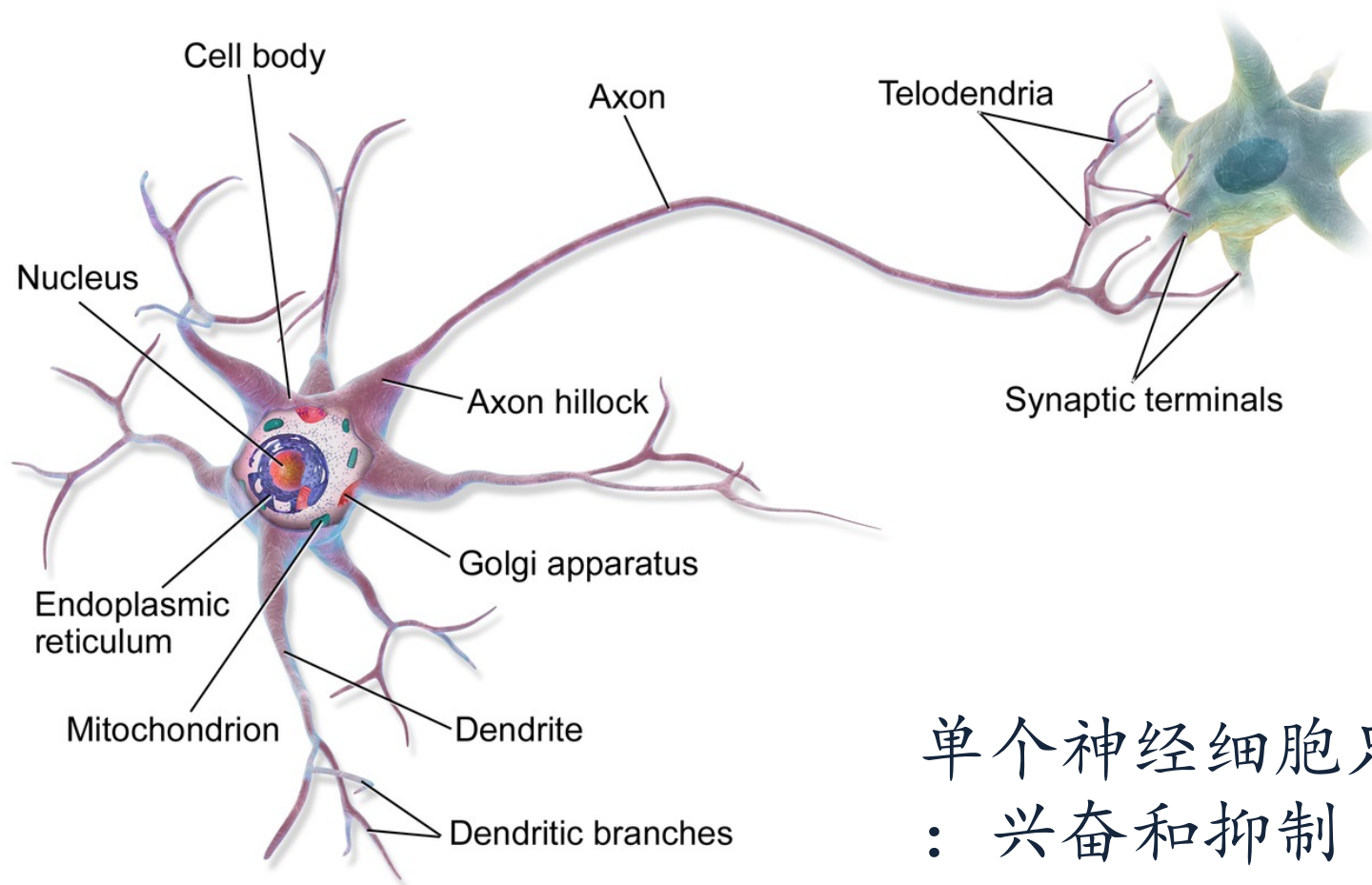
引入误差反向传播来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上。



神经元

生物神经元

[video: structure of brain](#)



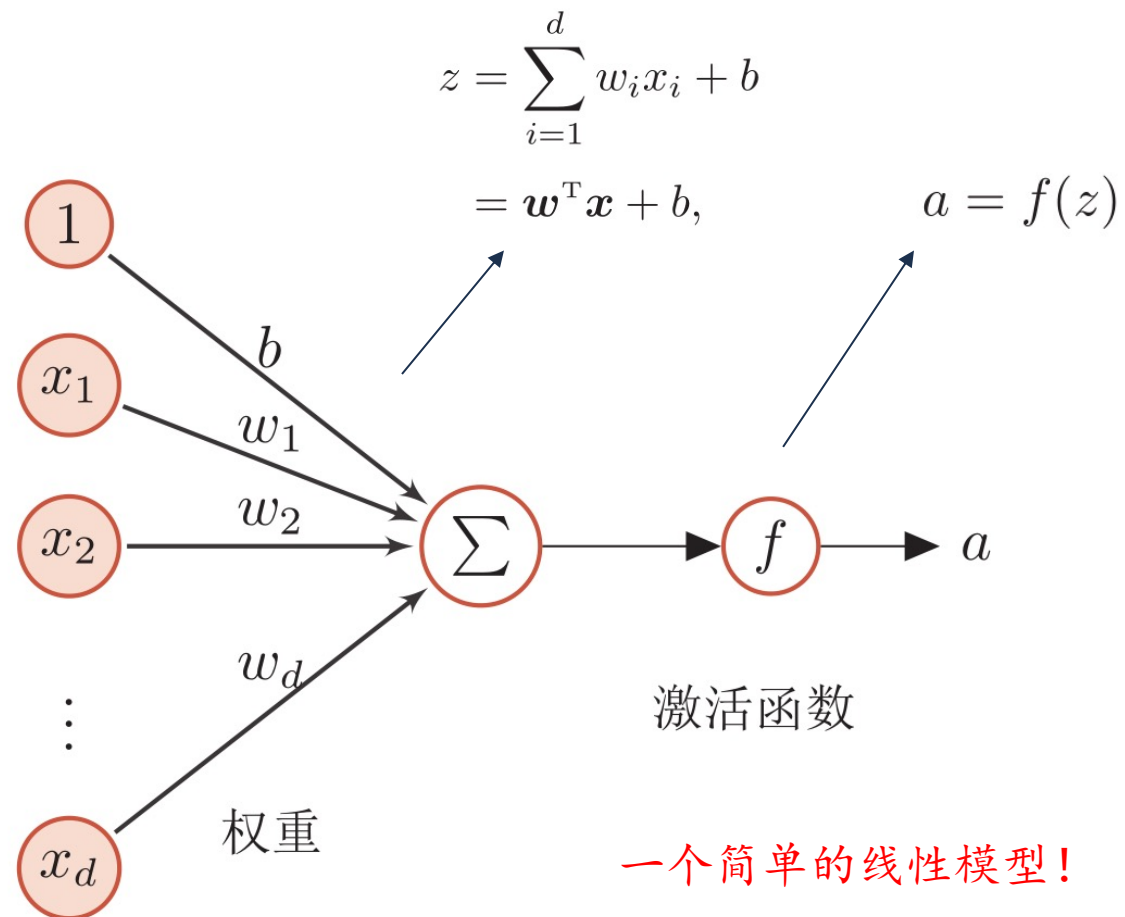
单个神经细胞只有两种状态：
兴奋和抑制

细胞突起是由细胞体延伸出来的细长部分，又可分为树突和轴突。

树突 (Dendrite) 可以接受刺激并将兴奋传入细胞体。每个神经元可以有一或多个树突。

轴突 (Axons) 可以把兴奋从胞体传送到另一个神经元或其他组织。每个神经元只有一个轴突。

人工神经元



激活函数的性质

连续并可导（允许少数点上不可导）的非线性函数。

可导的激活函数可以直接利用数值优化的方法来学习网络参数。

激活函数及其导函数要尽可能的简单

有利于提高网络计算效率。

激活函数的导函数的值域要在一个合适的区间内

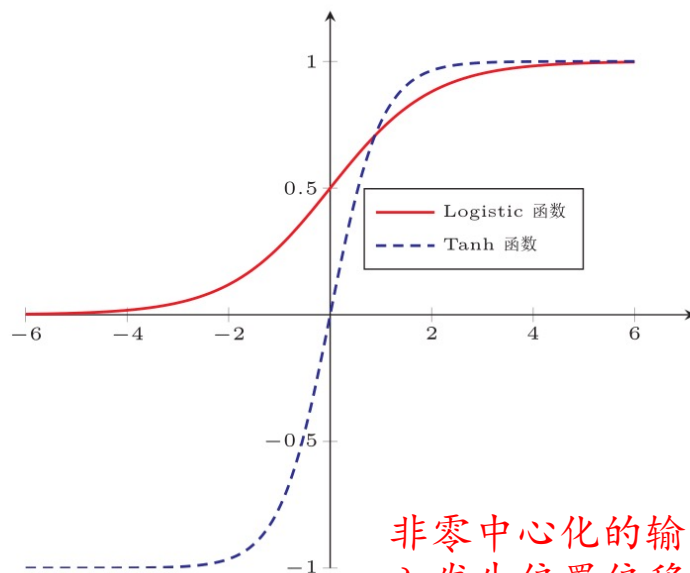
不能太大也不能太小，否则会影响训练的效率和稳定性。

单调递增

常见激活函数

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$



非零中心化的输出会使得其后的神经元的输入发生偏置偏移 (bias shift)，并进一步使得梯度下降的收敛速度变慢。

性质：

饱和函数

Tanh函数是零中心化的，而logistic函数（sigmoid 函数）的输出恒大于0

常见激活函数

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$= \max(0, x).$$

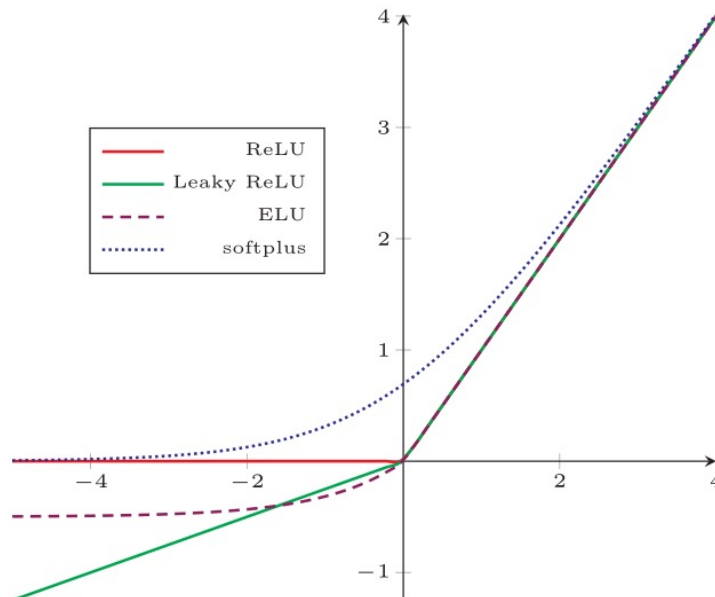
$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$

$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases}$$

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\text{softplus}(x) = \log(1 + \exp(x))$$



计算上更加高效

生物学合理性

单侧抑制、宽兴奋边界

在一定程度上缓解梯度消失问题

Dying ReLU Problem, 当激活输出小于0时

- 这个神经元的输出永远是 0
- 反向传播时梯度也永远是 0
- 权重、偏置不再更新

常见激活函数

► 高斯误差线性单元 (Gaussian Error Linear Unit, GELU)

$$\text{GELU}(x) = xP(X \leq x)$$

- 其中 $P(X \leq x)$ 是高斯分布 $N(\mu, \sigma^2)$ 的累积分布函数，其中 μ, σ 为超参数，一般设 $\mu = 0, \sigma = 1$ 即可
- 由于高斯分布的累积分布函数为S型函数，因此GELU可以用Tanh函数或Logistic函数来近似

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

或 $\text{GELU}(x) \approx x\sigma(1.702x).$

常见激活函数

► 高斯误差线性单元 (Gaussian Error Linear Unit, GELU)

$$\text{GELU}(x) = xP(X \leq x)$$

ReLU 是“硬门控”：

- $x > 0$ 保留
- $x < 0$ 直接砍掉

GELU 是“概率门控”：

- 输入越大，被保留的概率越高
- 小负值不会被完全抹掉
- 整个过程是平滑的

第一代 Transformer / 经典 LLM

应用：GELU 激活函数

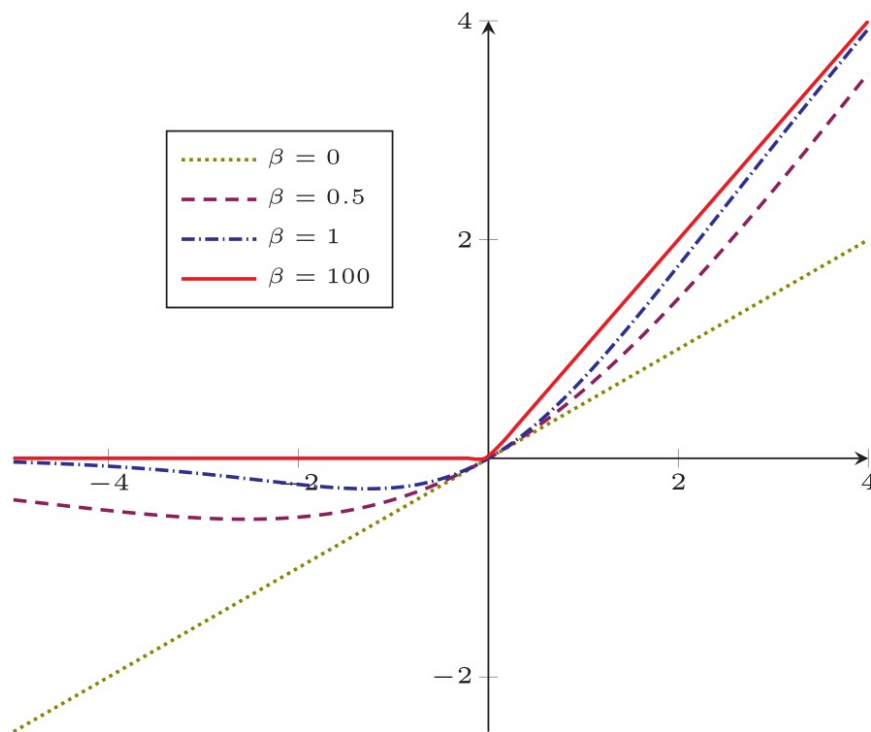
代表模型：

- BERT
- OpenAI 的 GPT-2 / GPT-3
- DeepMind 的很多早期 Transformer
- Vision Transformer (ViT)

常见激活函数

Swish函数

$$\text{swish}(x) = x\sigma(\beta x)$$



Swish或其门控变体用于
代表的现代 LLM模型:

- Meta 的 LLaMA 系列
- Mistral AI 的 Mistral
- PaLM 系列

在大模型里:

- 比 ReLU 稳定
- 梯度更平滑
- 对深层网络更友好

常见激活函数及其导数

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

人工神经网络

人工神经网络主要由大量的神经元以及它们之间的有向连接构成。因此考虑三方面：

神经元的激活规则

主要是指神经元输入到输出之间的映射关系，一般为非线性函数。

网络的拓扑结构

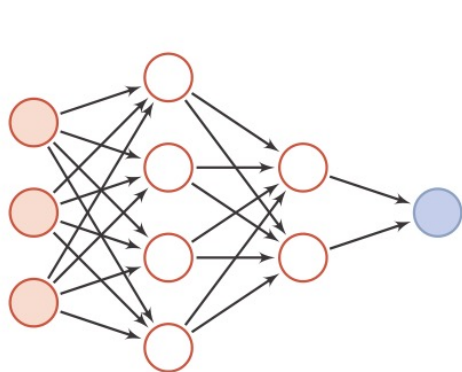
不同神经元之间的连接关系。

学习算法

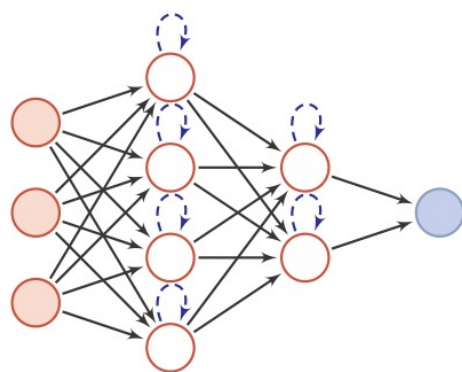
通过训练数据来学习神经网络的参数。

网络结构

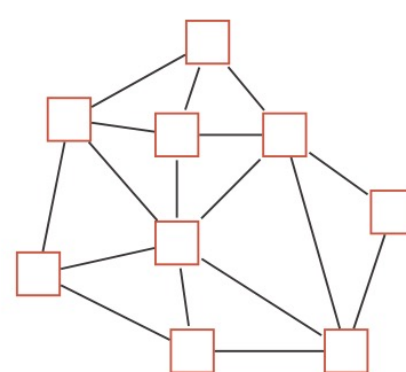
人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



(a) 前馈网络



(b) 记忆网络



(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。



前馈神经网络

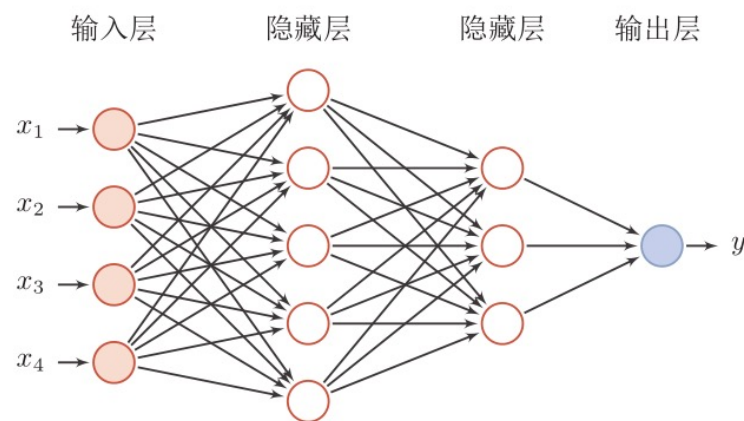
网络结构

前馈神经网络（全连接神经网络、多层感知器）

各神经元分别属于不同的层，层内无连接。

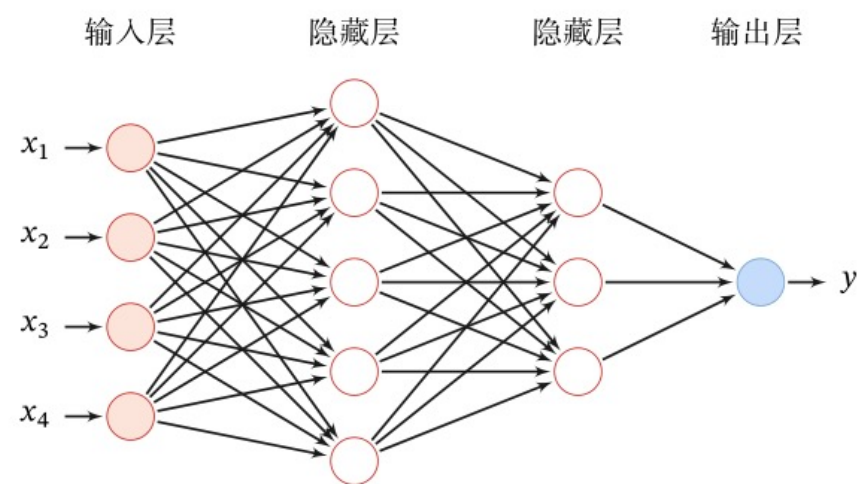
相邻两层之间的神经元全部两两连接。

整个网络中无反馈，信号从输入层向输出层单向传播，可用一个有向无环图表示。



前馈网络

给定一个前馈神经网络，用下面的记号来描述这样网络：



记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入（净活性值）
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出（活性值）

信息传递过程

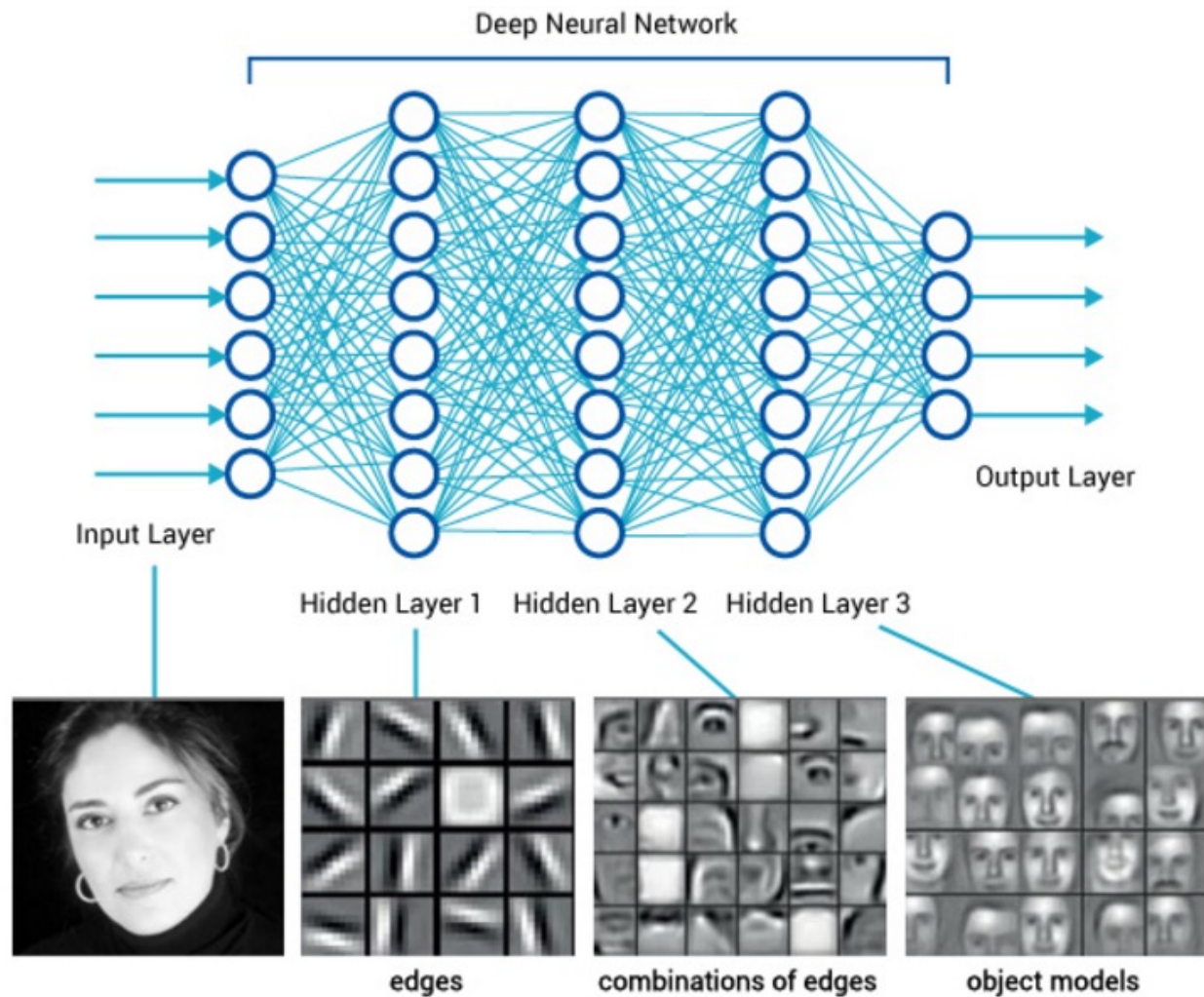
► 前馈神经网络通过下面公式进行信息传播。

$$\begin{aligned}\mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \\ \mathbf{a}^{(l)} &= f_l(\mathbf{z}^{(l)}).\end{aligned}$$

► 前馈计算：

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$

深层前馈神经网络



通用近似定理

定理 4.1 – 通用近似定理 (Universal Approximation Theorem)

[Cybenko, 1989, Hornik et al., 1989]: 令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数, $\mathcal{I}_d$ 是一个 d 维的单位超立方体 $[0, 1]^d$, $C(\mathcal{I}_d)$ 是定义在 $\mathcal{I}_d$ 上的连续函数集合。对于任何一个函数 $f \in C(\mathcal{I}_d)$, 存在一个整数 m , 和一组实数 $v_i, b_i \in \mathbb{R}$ 以及实数向量 $\mathbf{w}_i \in \mathbb{R}^d$, $i = 1, \dots, m$, 以至于我们可以定义函数

$$F(\mathbf{x}) = \sum_{i=1}^m v_i \varphi(\mathbf{w}_i^T \mathbf{x} + b_i), \quad (4.33)$$

作为函数 f 的近似实现, 即

$$|F(\mathbf{x}) - f(\mathbf{x})| < \epsilon, \forall \mathbf{x} \in \mathcal{I}_d. \quad (4.34)$$

其中 $\epsilon > 0$ 是一个很小的正数。

根据通用近似定理, 对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络, 只要其隐藏层神经元的数量足够, 它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。

应用到简单学习类任务

神经网络可以作为一个“万能”函数来使用，可以用来进行复杂的特征转换，或逼近一个复杂的条件分布。

$$\hat{y} = g(\varphi(\mathbf{x}), \theta)$$

分类器

神经网络

如果 $g(\cdot)$ 为Logistic回归，那么Logistic回归分类器可以看成神经网络的最后一层。



参数学习

应用到简单学习

对于多分类问题

如果使用Softmax回归分类器，相当于网络最后一层设置C个神经元，其输出经过Softmax函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

采用交叉熵损失函数，对于样本(x,y)，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

参数学习

给定训练集为 $D = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\hat{\mathbf{y}}^{(n)}$ ，其在数据集 D 上的结构化风险函数为：

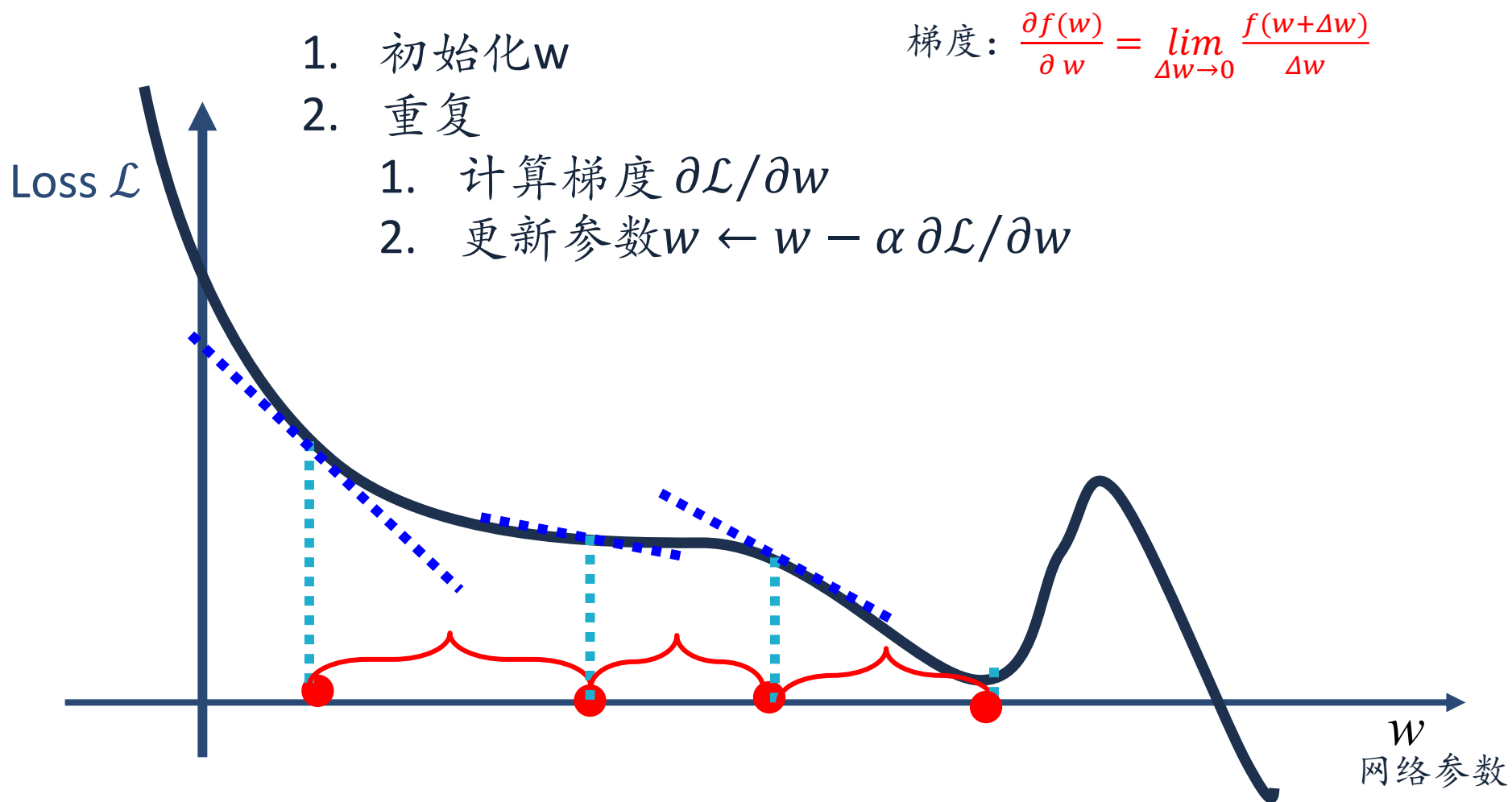
$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|W\|_F^2$$

梯度下降

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

梯度下降



如何计算梯度？

神经网络为一个复杂的复合函数

链式法则

$$y = f^5(f^4(f^3(f^2(f^1(x))))) \rightarrow \frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

反向传播算法

根据前馈网络的特点而设计的高效方法

一个更加通用的计算方法

自动微分 (Automatic Differentiation, AD)

矩阵微积分

矩阵微积分（Matrix Calculus）是多元微积分的一种表达方式，即使用矩阵和向量来表示因变量每个成分关于自变量每个成分的偏导数。

分母布局

标量关于向量的偏导数

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_p} \right]^T$$

向量关于向量的偏导数

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_q}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_p} & \dots & \frac{\partial y_q}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$

链式法则

链式法则（Chain Rule）是在微积分中求复合函数导数的一种常用方法。

(1) 若 $x \in \mathbb{R}$, $\mathbf{u} = u(x) \in \mathbb{R}^s$, $\mathbf{g} = g(\mathbf{u}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{g}}{\partial x} = \frac{\partial \mathbf{u}}{\partial x} \frac{\partial \mathbf{g}}{\partial \mathbf{u}} \in \mathbb{R}^{1 \times t}.$$

(2) 若 $\mathbf{x} \in \mathbb{R}^p$, $\mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^s$, $\mathbf{z} = f(\mathbf{y}) \in \mathbb{R}^t$, 则

$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{p \times t}.$$

(3) 若 $X \in \mathbb{R}^{p \times q}$ 为矩阵, $\mathbf{y} = g(X) \in \mathbb{R}^s$, $z = f(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial z}{\partial X_{ij}} = \frac{\partial \mathbf{y}}{\partial X_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$

反向传播算法

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}}, \end{aligned}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$$

误差项

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

计算 $\delta^{(l)} = \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{m^{(l)}}$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

前向传播：信息从输入层 $\rightarrow$ 输出层

反向传播：梯度/误差从输出层 $\rightarrow$ 输入层

$$\begin{aligned} \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} &= (W^{(l+1)})^T & \delta^{(l)} &\triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}} & \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} &= \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}} \\ & & & & &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \\ & & & & &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ & & & & &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (W^{(l+1)})^T \cdot \delta^{(l+1)} \\ & & & & &= f'_l(\mathbf{z}^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

反向传播算法

在计算出上面三个偏导数之后, 公式 (4.49) 可以写为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \mathbb{I}_i(a_j^{(l-1)})\delta^{(l)} = \delta_i^{(l)} a_j^{(l-1)}.$$

进一步, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层权重 $W^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial W^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top.$$

同理, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$



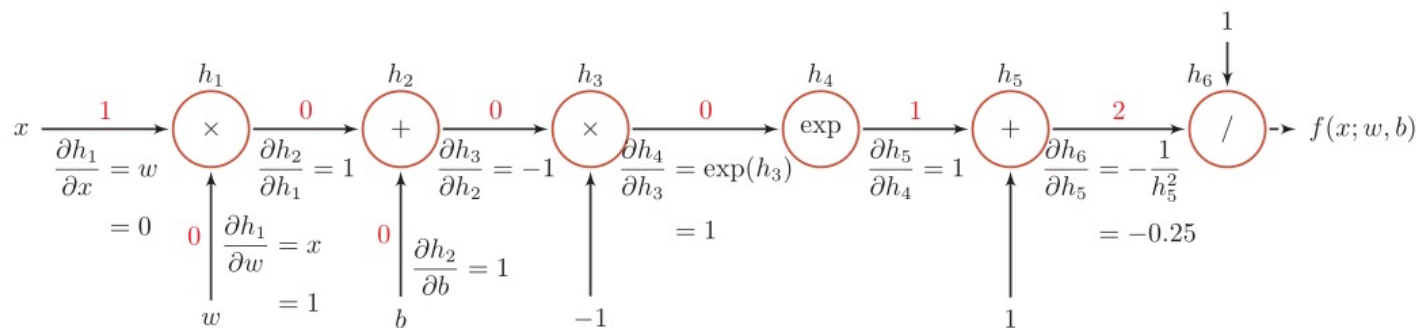
计算图与自动微分

计算图与自动微分

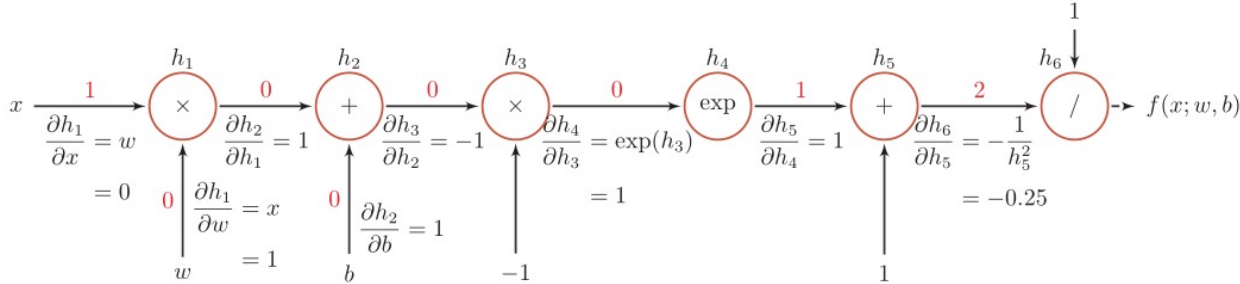
自动微分是利用链式法则来自动计算一个复合函数的梯度。

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

计算图



计算图



函数	导数	
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

当 $x = 1, w = 0, b = 0$ 时，可以得到

$$\begin{aligned}\frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25.\end{aligned}$$

静态计算图和动态计算图

静态计算图是在编译时构建计算图，计算图构建好之后在程序运行时不能改变。

如Tensorflow

动态计算图是在程序运行时动态构建。两种构建方式各有优缺点。

如PyTorch

静态计算图在构建时可以进行优化，并行能力强，但灵活性比较低。动态计算图则不容易优化，当不同输入的网络结构不一致时，难以并行计算，但是灵活性比较高。

动态计算图

PyTorch 中计算图本质是一个 有向无环图 (**DAG, Directed Acyclic Graph**) :

- **节点 (Node)** : 张量 (Tensor) 或 Function (操作)
- **边 (Edge)** : 操作的依赖关系

Pytorch中计算图反向传播本质是图的搜索/遍历:

1. 找到叶子节点 (Leaf Node)
2. 后序遍历
3. 梯度累加

自动微分

前向模式和反向模式

反向模式和反向传播的计算梯度的方式相同

如果函数和参数之间有多条路径，可以将这多条路径上的导数再进行相加，得到最终的梯度。

反向传播算法 (自动微分的反向模式)

前馈神经网络的训练过程可以分为以下三步

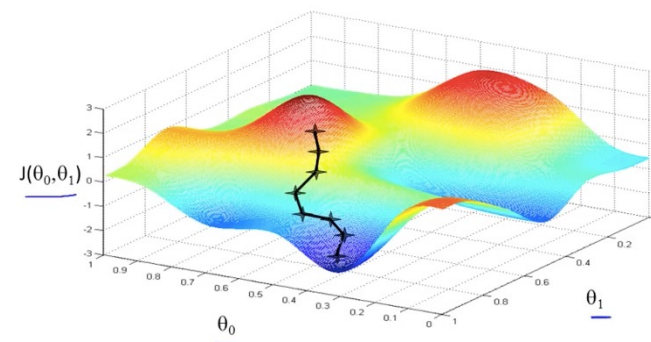
1. 前向计算每一层的状态和激活值，直到最后一层
2. 反向计算每一层的参数的偏导数
3. 更新参数

优化算法：随机梯度下降

算法 2.1 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

```
1 随机初始化  $\theta$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机排序;  
4   for  $n = 1 \cdots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6      $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$ ;           // 更新参数  
7   end  
8 until 模型  $f(\mathbf{x}; \theta)$  在验证集  $\mathcal{V}$  上的错误率不再下降;  
   输出:  $\theta$ 
```



优化算法：小批量随机梯度下降 MiniBatch

选取 K 个训练样本 $\{\mathbf{x}^{(k)}, \mathbf{y}^{(k)}\}_{k=1}^K$ ，计算偏导数

$$\mathbf{g}_t(\theta) = \frac{1}{K} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{S}_t} \frac{\partial \mathcal{L}(\mathbf{y}, f(\mathbf{x}; \theta))}{\partial \theta}$$

定义梯度

$$\mathbf{g}_t \triangleq \mathbf{g}_t(\theta_{t-1})$$

$$\theta_t \leftarrow \theta_{t-1} - \alpha \mathbf{g}_t$$

更新参数

几个关键因素：

- 小批量样本数量
- 梯度
- 学习率

其中 $\alpha > 0$ 为学习率

深度学习的三大步骤





优化问题

优化问题

难点

参数过多，影响训练

非凸优化问题：即存在局部最优而非全局最优解，影响迭代

梯度消失问题，下层参数比较难调

参数解释起来比较困难

需求

计算资源要大

数据要多

算法效率要好：即收敛快

优化问题

非凸优化问题

什么是非凸优化？

如果一个优化问题的目标函数是“碗形”的（只有一个最低点），叫凸优化。

如果函数形状复杂、起伏很多，有多个极小点、鞍点、平台区 —— 就是非凸优化。

深度神经网络的训练目标几乎都是非凸的。

非凸优化的根本问题是：

损失函数地形复杂 → 无法保证找到全局最优 → 优化路径强依赖初始化与动态噪声。

但在深度学习中：

并不追求“全局最优”，而追求“足够好且泛化好”的解

优化问题

梯度爆炸问题 (Gradient Problem)

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

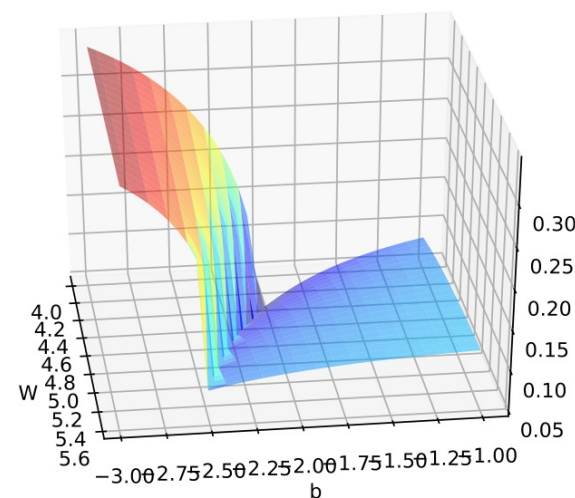
如果网络层数很深，或者权重、激活函数不合适，这些链式乘积可能变得非常大。

策略1：梯度截断

梯度截断是一种比较简单的启发式方法，把梯度的模限定在一个区间，当梯度的模小于或大于这个区间时就进行截断。

按值截断 $\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$

按模截断 $\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|} \mathbf{g}_t$



策略2：参数初始化

基于方差缩放的参数初始化

Xavier 初始化和 He 初始化

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

正交初始化

- 1) 用均值为 0、方差为 1 的高斯分布初始化一个矩阵；
- 2) 将这个矩阵用奇异值分解得到两个正交矩阵，并使用其中之一作为权重矩阵。

策略3：数据预处理

数据归一化

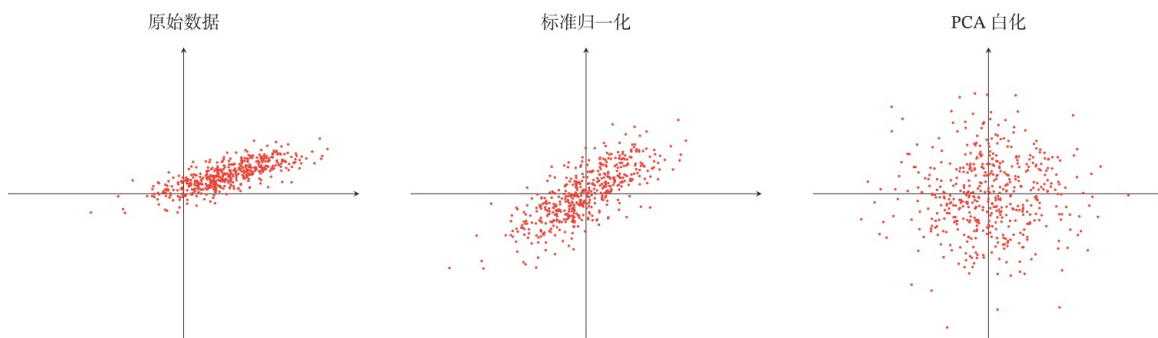
最小最大值归一化

标准化

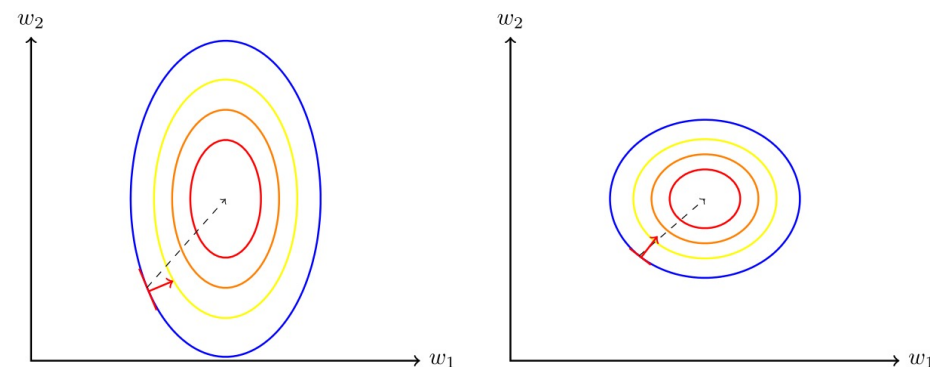
PCA



$$\mu = \frac{1}{N} \sum_{n=1}^N x^{(n)},$$
$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2.$$



数据归一化对梯度的影响



(a) 未归一化数据的梯度

(b) 归一化数据的梯度

批量归一化 (Batch Normalization)

对于一个深层神经网络，令第 l 层的净输入为 $\mathbf{z}^{(l)}$ ，神经元的输出为 $\mathbf{a}^{(l)}$ ，即

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) = f(W\mathbf{a}^{(l-1)} + \mathbf{b}), \quad (7.42)$$

其中 $f(\cdot)$ 是激活函数， W 和 $\mathbf{b}$ 是可学习的参数。

给定一个包含 K 个样本的小批量样本集合，计算均值和方差

$$\begin{aligned} \mu_{\mathcal{B}} &= \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)}, \\ \sigma_{\mathcal{B}}^2 &= \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}). \end{aligned}$$

批量归一化

$$\begin{aligned} \hat{\mathbf{z}}^{(l)} &= \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta \\ &\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}), \end{aligned}$$